

The widely tabulated lower bound $S(13)$ 44 rests on an invalid proof: an audit of van Voorhis (1972), a counterexample, and a partial repair

INTERNAL DRAFT — not for submission or circulation. Version 0.1, 2026-08-18. Editorial notes, open queries and flagged source discrepancies are in docs/paper/NOTES.md.

Abstract

The minimum size $S(13)$ of a 13-channel sorting network is unknown. Since April 2025 the most widely consulted table of sorting-network bounds has listed $S(13) \geq 44$, and that value has propagated into secondary summaries. It is obtained as $S(11) + P(2,13) = 35 + 9$, where the second summand comes from a “two-value pruning” theorem of van Voorhis, published in 1972 as a chapter of *Complexity of Computer Computations*.

We audit that chapter’s proof. Two of its structural steps — equations (5) and (6) — are false. We exhibit an explicit, size-optimal 3-comparator 3-sorter on which equation (5) over-states the quantity it claims to compute (it returns 4 where the truth is 3) and equation (6)’s Kraft sum equals $5/4 > 1$, which is the direction that destroys the derivation. Equation (7) is derived from (5) and (6) and from nothing else, and (8), (10), (12) are derived from (7). We trace the fault to a single false premise on p. 121, the claim that the MAX subnetwork of an N-sorter “includes N-1 comparators”: *pass-through* comparators, traversed by a max-path but not branching it, are ignored. Pass-throughs occur in 236 of 387 audited sorters (61.0 %). We record ten steelman defences of the chapter, including the comparator-disjointness lemma that a sympathetic reader reconstructs first, and refute each. We separate a genuine misstatement in the chapter’s own worked example (Fig. 4: the Kraft sum is 0.75, not 1) from the load-bearing failure (a sum exceeding 1), because only the latter breaks the argument.

We then repair the theorem for a large class. Calling an N-sorter *clean* when its MAX subnetwork contains no pass-through, we give a complete proof of equation (8) for clean sorters, based on a two-colouring of wire segments (red / blue leads) that supplies the antichain step van Voorhis assumed and weakens his Kraft *equality* to the *inequality* the argument needs. We also close off the natural general repair: the per-node charging lemma that implies (8) is refuted by an explicit 15-comparator 6-sorter, so any valid proof must be global rather than term-by-term.

We find no erratum, correction or reproof of the chapter in the 54 years since publication, and no modern citation that engages with the two-value argument at all. $S(13) \geq 44$ has never appeared in a peer-reviewed publication; the best *published* lower bound is 43, and it is unaffected. Our conclusion is narrow and

we state it as such: the two-value theorem is neither proved nor refuted, it is an open conjecture with substantial empirical support (zero violations across 1,890 constructed sorters and two exhaustive small- n enumerations), and the tabulated 44 is currently not supported by a correct argument.

Every mathematical claim in this paper is keyed to a check in one of two independent stdlib-only Python scripts, `tools/verify_huffman2.py` (48 checks) and `tools/verify_kraft_dispute.py` (36 checks), both of which exit 0.

1. Introduction

1.1 The quantity

A *comparator* (a, b) with $a < b$ on a set of n numbered channels replaces the pair of values on channels a and b by their minimum on a and their maximum on b . A *sorting network* on n channels, or n -sorter, is a finite sequence of comparators that sorts every input; $S(n)$ denotes the least number of comparators in an n -sorter. $S(n)$ is known exactly only for $n \leq 12$:

n	1	2	3	4	5	6	7	8	9	10	11	12	13
$S(n)$	0	1	3	5	9	12	16	19	25	29	35	39	?

The values through $n = 8$ are due to Floyd and Knuth; $S(9) = 25$ and $S(10) = 29$ were settled by Codish, Cruz-Filipe, Frank and Schneider-Kamp; $S(11) = 35$ and $S(12) = 39$ by Harder in 2020. For $n = 13$ the best known upper bound is 45, from a network found by Juillé in 1995 and never improved. (Van Voorhis's own 1972 table records $U(13) = 46$, the then-best construction, attributed to Green.)

1.2 The two competing lower bounds

Two lower-bound arguments are in circulation.

The one-value bound. $S(n) \geq S(n-1) + \text{ceil}(\log_2 n)$. This is van Voorhis's result in the *IEEE Transactions on Computers* note of 1972; it is Knuth's exercise 5.3.4–42 in *The Art of Computer Programming* Vol. 3, with a three-line proof given in the answers. At $n = 13$ it yields

$$S(13) \geq S(12) + \text{ceil}(\log_2 13) = 39 + 4 = 43.$$

This is the best *published* lower bound and nothing in this paper disturbs it.

The two-value bound. $S(N) \geq S(N-2) + P(2, N)$, together with $P(2, N) \geq \text{ceil}(\log_2 F(N))$ for an explicitly computable F . This is the content of the Plenum chapter [VV72]. With $F(13) = 392$ and $\text{ceil}(\log_2 392) = 9$, and the modern $S(11) = 35$, it yields

$$S(13) \geq S(11) + P(2,13) = 35 + 9 = 44.$$

The chapter itself printed $L(13) = 42$ in its Table 1, because in 1972 the best available $S(11)$ figure was 33. The 44 is a modern recombination: van Voorhis’s $P(2,13) = 9$ plus Harder’s 2020 value of $S(11)$. It appears to have entered the public record on 2025-04-21 through a changelog entry in Dobbelaere’s sorting-network table, “Tighter lower bounds for size, on suggestion of Jelmer Fiet and based on principles in [V Voorh72]”. We could locate no publication or preprint behind it (§6).

1.3 What this paper claims

1. **The chapter’s proof of $P(2,N) \geq \text{ceil}(\log_2 F(N))$ is invalid as written.** Equations (5) and (6) are false, under every reading of the chapter’s own definitions that we could construct, and equation (7) — the only route from them to the result — is derived from them and from nothing else (§4).
2. **The failure is common, and it is not confined to slack.** Pass-through comparators occur in 61.0 % of an audited population of 387 constructed sorters; equation (6) fails in the fatal direction in 42.4 %; and 27.1 % of the population simultaneously has zero slack in the conclusion and a broken Kraft sum (§4.6).
3. **The natural defences all fail**, including the MAX/MAX2 comparator-disjointness lemma that would repair both equations at one stroke, and which is almost certainly what the chapter had in mind (§5).
4. **No erratum exists, and the 44 was never peer-reviewed (§6).**
5. **A complete, correct proof exists for clean sorters** — those whose MAX subnetwork has no pass-through — via a red/blue colouring of leads. This is, as far as we can establish, the first correct proof of any case of the 1972 theorem (§7).
6. **The general case cannot be repaired by per-node charging.** The strongest member of that family of repairs is refuted by an explicit 15-comparator 6-sorter (§7.6).
7. **Status.** The two-value theorem is an open conjecture. It survives 1,890 constructed sorters and two exhaustive small- n enumerations with zero violations, and it is consistent with every exact $S(n)$ known (§8).

We claim nothing about the truth of $S(13) \geq 44$. We claim that it is not currently supported by a correct argument, while $S(13) \geq 43$ is.

1.4 Artifact

Two independent Python programs accompany this paper. `verify_huffman2.py` (48 checks) was written alongside the first reading of the chapter; `verify_kraft_dispute.py` (36 checks) was written afterwards, from the page images, as an adversarial re-derivation whose brief was to defend the chapter, and it shares no code with the first. Both are pure standard library, read-only, deterministic under fixed

seeds, and exit 0. §9 gives exact commands, hashes and a claim-to-check index.

Citation convention. Individual checks are cited inline as **V1:A1**, **V2:C5** and so on. The two scripts number their checks independently and the IDs collide — each defines an **A1**, a **B1**, a **C1**, a **D1**, an **E3** — so the prefix is load-bearing: **V1:** means `verify_huffman2.py` and **V2:** means `verify_kraft_dispute.py`.

2. Preliminaries

We fix conventions matching the verifier code, since several of the distinctions below are exactly what the 1972 argument elides.

Networks. A network on n channels $0..n-1$ is a list of comparators (a_t , b_t), $a_t < b_t$, applied in order; each writes the minimum of its two inputs to a_t and the maximum to b_t . A network *sorts* if it sorts every 0/1 input (the 0/1 principle); every network in the artifact is checked this way. For a sorting network the maximum output o_N is channel $n-1$, the second largest o_{N-1} is channel $n-2$, the minimum o_1 is channel 0.

Leads. A *lead* is a wire segment: the input segment of a channel, or one of the two output segments of a comparator. The artifact numbers leads explicitly (`lead_ids`), giving each comparator a *low output lead* and a *high output lead*.

Max-paths. Assign to input channel k a value strictly greater than all others. It leaves every comparator it enters on the high output, so it follows a unique path p_k of comparators from channel k to o_N . $MAX(T)$ is the set of comparators lying on some p_k .

Branch nodes, pass-throughs, the branch tree. A comparator c is a *branch node* iff max-paths arrive at c on *both* of its input leads. A comparator in $MAX(T)$ that is not a branch node is a *pass-through*: a max-path enters and leaves it on the same channel, and no other max-path arrives on the other lead. The branch nodes, ordered by the merging structure of the max-paths, form the *branch tree* B : n leaves (the input channels) and $n-1$ internal nodes, rooted at the last branch node before o_N . In the artifact the branch tree is built from the comparator sequence, not from path strings, because two channels can share an entire path string (`_build`); the construction asserts n leaves and $n-1$ branch nodes and has never failed on any network tested (check **V1:B1**).

Definition (clean). An N -sorter is **clean** if $|MAX(T)| = N-1$, equivalently if it has no pass-through comparator, equivalently if every comparator its max-paths traverse is a branch node.

The weight function. For a branch node c with branch-tree subtrees $L(c)$, $R(c)$ of heights $lp(L(c))$, $lp(R(c))$, and depth $depth_B(c)$ (root at depth 0), set

$$a(c) := lp(L(c)) + lp(R(c)) + depth_B(c) + 1, \quad f(B) := \sum_c 2^{\{a(c)\}}.$$

f so defined satisfies van Voorhis's Theorem 1 recursion $f(B) = 2[f(L(B)) + f(R(B)) + 2^{\lfloor \lg(L(B)) + \lg(R(B)) \rfloor}]$ and reproduces his $F(N) = \min_B f(B)$ column of Table 1 exactly for $N \leq 16$ (checks **V1:A1**, **V1:A2**; independently, under a separate implementation, **V2:A1**):

$F(N)$, $N = 1..16$: 0, 2, 8, 16, 36, 52, 80, 96, 168, 200, 256, 288, 392, 424, 480, 512

Pruning. For an ordered pair (x, y) of distinct input channels, run the scenario *max at x, second max at y, all other inputs smaller*, and let $W(x, y)$ be the set of comparators touched by either of the two values. Then $p(2, T) := \max_{\{x, y\}} |W(x, y)|$, and removing $W(x, y)$ from T leaves an $(N-2)$ -sorter, so $|T| \geq S(N-2) + p(2, T)$. The residual claim is verified directly by path contraction on every network tested (check **V1:B5**), and $p(2, T)$ is computed by brute force over all ordered pairs (**brute_p2**).

Two readings of nc. The chapter defines $nc(C_j)$ as a count of actual comparators (p. 122). We call this the *literal* reading, **nc_actual**. The *tree* reading is **a(c)** above. Always $nc_actual(c) \geq a(c)$, with equality iff no pass-through lies on the relevant paths. The two differ on 27 of the 43 structured networks of the artifact's Part B. The distinction is load-bearing (§4.5).

3. The 1972 argument

We restate the chapter's derivation in its own numbering, quoting minimally. All page numbers are book pages of [VV72] (PDF pages 124–134 of the scanned volume in **papers/**).

The pruning idea is credited to Green (p. 120). For k inputs given the k largest values, “[p]runing the comparators traversed by these paths separates the N -sorter into k wires and an $(N-k)$ -sorter” (p. 121), and with $P(k, N) = \min_T p(k, T)$ (eq. 1),

$$(2) \quad S(N) \geq S(N-k) + P(k, N).$$

For $k = 1$ the chapter obtains $P(1, N) = \lceil \log_2 N \rceil$ (eq. 3) from the structural claim on p. 121:

“It has been observed [Van Voorhis (1972A)] that the MAX subnetwork of any N -sorter is a binary tree with N leaves (the input leads) and $N-1$ branch nodes (the comparators) rooted at o_N .” — p. 121

and restates it immediately at the head of the section bounding $P(2, N)$:

“The MAX subnetwork of T , $MAX(T)$, includes $N-1$ comparators, which we label $c_1, c_2, \dots, c_{N-1}$.” — p. 121

The bracketed attribution [Van Voorhis (1972A)] is to the IEEE Transactions note (§11). A footnote on p. 121 adds only: “In this section we assume several properties of binary trees which are derived, for example, in Knuth(1968A).”

For a branch node c_j , $L(C_j)$ and $R(C_j)$ denote the two subtrees feeding its inputs; p_{j1} , p_{j2} are the longest max-paths through them; and

“We define $\text{nc}(\mathbf{c}_j)$ to be the number of comparators from $\mathbf{i}_{\{j1\}}$ to $\mathbf{c}_j$, plus the number from $\mathbf{i}_{\{j2\}}$ to $\mathbf{c}_j$, plus the number from $\mathbf{c}_j$ to $\mathbf{o}_N$. (Comparator $\mathbf{c}_j$ itself is counted exactly once.)” — p. 122

Placing the largest value at a leaf of $L(C_j)$ and the second largest at a leaf of $R(C_j)$, the two meet at c_j ; the largest leaves high and proceeds to o_N ; the second largest leaves low, so

“T must include a path q_j from the lower output lead of c_j to $o_{\{N-1\}}$. The paths q_j , $1 \leq j \leq N-1$, together form a binary tree rooted at $o_{\{N-1\}}$, which we call the MAX2 subnetwork.” — p. 122

and then the two steps we dispute:

“... then the two paths from i_{j1} and i_{j2} through c_j to o_N and o_{N-1} together include $nc(C_j) + nc(q_j)$ comparators. Therefore, $p(2, T) = \max_{1 \leq j \leq N-1} [nc(C_j) + nc(q_j)]$.” — p. 122, eq. (5)

“MAX2(T) is a binary tree, so the path lengths $nc(q_j)$, $1 \leq j \leq N-1$, satisfy $\sum_{1 \leq j \leq N-1} 2^{-nc(q_j)} = 1$.” — p. 124, eq. (6)

From these:

(7) “Equation (5) implies that $\text{nc}(\mathbf{q}_j) \leq p(2, T) - \text{nc}(\mathbf{C}_j)$, so that $\sum_{1 \leq j \leq N-1} 2^{p(2, T) - \text{nc}(\mathbf{C}_j)} \leq 1$.” — p. 124, eq.

“Since $p(2, T)$ is integral, we conclude that $p(2, T) \geq \text{ceil}(\log_2(f(\text{MAX}(T))))$, where $f(\text{MAX}(T)) = \sum_{1 \leq j \leq N-1} 2^{+nc(C_j)}$.” — p. 124, eqs. (8), (9)

and, since the bound depends only on the structure of $\text{MAX}(\mathbf{T})$, minimising over binary trees gives (10), (11) and

$$(12) \quad P(2,N) \geq \text{ceil}(\log_2 F(N)), \quad F(N) = \min_B f(B).$$

Theorem 1 (eq. 13, p. 125) supplies the recursion for F , whose proof (eqs. 14–18) computes $nc(c_j)$ by adding 1 for each step toward the root — that is, it treats nc as a *tree depth*. Table 1 (p. 128) then lists $P(2, 13) = 9$ and $L(13) = 42$.

The load-bearing chain, isolated.

```

(6) MAX2 Kraft = 1      (5) p(2,T) = max_j [nc(c_j) + nc(q_j)]
      \
      -----> (7) <-----
                |
                (8) p(2,T) >= ceil(log2 f(MAX(T)))
                |

```

$$\begin{array}{c}
(10), (11), (12) \quad P(2,N) \geq \text{ceil}(\log_2 F(N)) \\
| \\
P(2,13) \geq \text{ceil}(\log_2 392) = 9, \quad S(13) \geq S(11) + 9 = 44
\end{array}$$

Note what (7) actually requires: only $\sum_j 2^{-nc(q_j)} \leq 1$, and only $nc(c_j) + nc(q_j) \leq p(2,T)$. The equalities asserted in (5) and (6) are stronger than the derivation needs. This matters for §4.7: an equality can fail harmlessly. The failures we exhibit are in the fatal direction anyway.

We also record, for §8, the chapter’s one place where the F -bound is beaten:

“For all values of $N \leq 16$ except $N = 11$, we have constructed an N -sorter T satisfying $p(2,T) = \text{ceil}(\log_2(F(N)))$. However, we have been able to show that $P(2,11) = 9$ as follows. Our bound for $p(2,T)$ is derived under the assumption that we prune T by removing paths followed by the largest two input values to o_N and o_{N-1} . Clearly it works equally well to prune paths followed by the smallest two input values to o_1 and o_2 . It turns out that if T is an 11-sorter such that $f(\text{MAX}(T)) = F(N) = 256$, then we can prune two paths to o_1 and o_2 that together include 9 comparators.” — p. 127

No proof of the $N = 11$ claim is given.

4. The audit

4.1 The counterexample

Let

$T1 = [(0,1), (0,2), (1,2)]$ on channels 0,1,2

which is the 3-sorter produced by Batcher’s construction. $|T1| = 3 = S(3)$ and it sorts (checks **V1:E0** / **V2:C0**). Its max-paths are

max entered on	comparators traversed	ends at
ch 0	$c_0 = (0,1), c_2 = (1,2)$	ch 2
ch 1	$c_0 = (0,1), c_2 = (1,2)$	ch 2
ch 2	$c_1 = (0,2), c_2 = (1,2)$	ch 2

Everything below is recomputed independently by both verifiers, and is small enough to check by hand.

4.2 The false premise: $\text{MAX}(T)$ need not have $N-1$ comparators

$\text{MAX}(T1) = \{c_0, c_1, c_2\}$ — **three** comparators, where p. 121 asserts $N-1 = 2$. The comparator $c_1 = (0,2)$ is a **pass-through**: the maximum enters

and leaves it on channel 2, and no other max-path arrives on channel 0 at that point, so c_1 lies on the MAX subnetwork but is not a branch node. The branch nodes are $\{c_0, c_2\}$ and there are indeed $N-1 = 2$ of them (checks **V1:E1**; **V2:C1**, **V2:C2**).

The salvageable half of the chapter’s claim is that the *branch* tree has N leaves and $N-1$ nodes; that survives every test (check **V1:B1**). What does not survive is the identification of branch nodes with the comparators of $\text{MAX}(T)$, and hence the identification of nc with a tree depth, which Theorem 1’s own proof (eqs. 14–18) silently requires.

We regard the single sentence “ $N-1$ branch nodes (the comparators)” as the proximate error. Everything downstream inherits it.

4.3 Equation (5) is false, and in the fatal direction

Take the branch node c_0 with realising pair $(i_{\{j1\}}, i_{\{j2\}}) = (ch0, ch1)$:

- the max path is c_0, c_2 ;
- the second-max path is c_0, c_1, c_2 ;
- $nc(c_0) = 0 + 0 + 2 = 2$ and $nc(q_0) = 2$, so eq. (5) asserts that the two paths “together include” $2 + 2 = 4$ comparators.

They together include **three**: the union is $\{c_0, c_1, c_2\}$, because after separating at c_0 the max and the second max **meet again** at c_2 . Since $|T1| = 3$, the true $p(2, T1) = 3$ is forced, and brute force over all ordered pairs confirms it.

So eq. (5) returns 4 for a quantity whose value is 3. It **exceeds** $p(2, T)$, which is precisely the direction that invalidates (7): (7) needs $nc(q_j) \leq p(2, T) - nc(c_j)$, and eq. (5) is the only thing offered in support (checks **V1:E2**, **V1:E3**; **V2:C3**, **V2:C4**).

The general phenomenon is exact and we name it: writing $ov(c)$ for the number of comparators traversed after c by *both* values,

$$|W(x, y)| = nc(c) + nc(q_c) - ov(c),$$

and eq. (5) is the claim $ov(c) = 0$. In the artifact’s structured sweep there are 117 $(c_j, i_{\{j1\}}, i_{\{j2\}})$ triples on which the two paths re-meet.

4.4 Equation (6) is false, and in the fatal direction

The MAX2 “leaves” of $T1$ are the low output leads of c_0 and c_2 . Their depths are $nc(q_0) = 2$ and $nc(q_2) = 0$ — the low output of c_2 is $o_{\{N-1\}}$. Hence

$$\sum_j 2^{-nc(q_j)} = 2^{-2} + 2^{-0} = 5/4 = 1.25 > 1$$

so MAX2 is not a binary tree with $N-1$ leaves and eq. (6) fails (checks **V1:E4**; **V2:C5**). The root cause is structural, not arithmetic: q_0 *passes through* the

low output lead of c_2 , so the claimed leaves are not an antichain — one of them lies on the path from another (check **V1:B3**).

This is the fatal direction. Step (7) needs $\sum_j 2^{-nc(q_j)} \leq 1$; a sum below 1 is harmless, a sum above 1 is not.

4.5 Equation (9) read literally contradicts equation (8)

With nc computed exactly as the chapter defines it on p. 122 — actual comparator counts — T1 gives $nc(c_0) = 2$, $nc(c_2) = 3$, so $f = 2^2 + 2^3 = 12$ and eq. (8) asserts $p(2, T1) \geq \lceil \log_2 12 \rceil = 4$, contradicting $p(2, T1) = 3$. **Under the literal reading, equation (8) itself is false** (checks **V1:E5**; **V2:C6**, **V2:C7**).

With f computed by Theorem 1 on the branch tree, $f = 2^2 + 2^2 = 8$ and (8) asserts $p(2, T1) \geq 3$, which holds with equality (checks **V1:E6**; **V2:C8**).

In the randomised sweep the literal reading of (8) is violated by 90 of 560 networks (16 %) in the first verifier and by 102 of 387 (26.4 %) in the second; the tree reading is violated by none in either.

Consequence. f must be read as Theorem 1’s recursive function of the branch tree, not as eq. (9)’s literal sum. This reading is forced independently by Fig. 5, which is captioned “Symmetric MAX Subnetwork” — a redrawn tree — and whose parenthesised nc values (3), (3), (4), (3), (3), (4), (5) are exactly the branch-tree quantities (checks **V1:E7**, **V2:B1**). We adopt the tree reading throughout, and it is the reading under which the chapter’s conclusion is empirically unrefuted. It does not rescue (5) or (6).

4.6 Prevalence: the failure is common, and it is not confined to slack

Two independently written sweeps agree on the order of magnitude. The second verifier’s population is 387 constructed networks (bubble, insertion, odd-even transposition and Batcher for $n = 3 \dots 10$; randomised prefix-plus-thinning for $n = 3 \dots 8$; random channel relabellings), each verified a sorter by the 0/1 principle:

statement	count	share
MAX(T) has at least one pass-through	236	61.0 %
some q_j re-meets the max path after c_j	233	60.2 %
eq. (5) is not an equality	172	44.4 %
eq. (5) RHS strictly exceeds $p(2, T)$ (fatal)	172	44.4 %
eq. (6) is not an equality	233	60.2 %

statement	count	share
eq. (6) $\text{sum} > 1$ (fatal)	164	42.4 %
MAX2(T) shares a comparator with MAX(T)	233	60.2 %
MAX2(T) shares a branch node with MAX(T)	232	59.9 %
eq. (7) fails, literal nc	102	26.4 %
eq. (8) fails, literal nc	102	26.4 %
eq. (7) fails, tree nc	0	0.0 %
eq. (8) fails, tree nc	0	0.0 %
conclusion $p(2, T) \geq \text{ceil}(\log_2 F(n))$ fails	0	0.0 %
$p(2, T) = \text{ceil}(\log_2 f_tree)$ exactly, i.e. no slack	220	56.8 %
no slack and eq. (6) fails with $\text{sum} > 1$	105	27.1 %

The first verifier’s independent population of 560 randomised sorters ($n = 4..10$) gives eq. (6) violated in 192 (34 %) and the literal eq. (9) violated in 90 (16 %) (check **V1:F4**).

The last row is the answer to the natural dismissal, “surely this is a harmless technicality that only bites where the bound has slack to spare”. It is not: 27.1 % of the population has the bound tight *and* the Kraft step broken. The broken lemma fails precisely in the regime where the theorem is extremal (check **V2:D0**).

A note on what is asserted versus what is printed. The table above is printed by the second verifier’s Part D. The registered checks over that sweep assert existence and universality statements rather than the individual counts: **V2:D1** asserts that eq. (5) and eq. (6) each fail on a positive fraction, **V2:D0** that at least one failure is tight, **V2:D-1** that disjointness fails somewhere, **V2:D-2** that no pass-through-free network fails, and **V2:D2/V2:D3** that the tree reading of eq. (8) and the conclusion never fail. The percentages themselves are reproducible output of a deterministic seeded run, not asserted invariants, and they are properties of these generators (see §11, item 3). The same caveat applies to the first verifier’s “27 of 43” pass-through count and “12 of 43” eq. (5) over-statement count, which are printed statistics rather than registered checks; the corresponding *existence* claims are checks **V1:B2**, **V1:B3**, **V1:B4b** and **V1:F4**.

4.7 Fig. 4: a misstatement, not a gap — the distinction matters

The chapter’s only worked example is Batchner’s odd-even 8-sorter (Figs. 3–5), reconstructed from the standard algorithm rather than from any table. Its $\text{MAX}(\mathbf{T})$ has 7 comparators, 7 branch nodes and **no pass-throughs**; its nc multiset is $\{3, 3, 3, 3, 4, 4, 5\}$, exactly the numbers printed in Fig. 5; and $f(\text{MAX}(\mathbf{T})) = 96 = F(8)$ (checks **V1:E7**; **V2:B1**, **V2:B2**). But the MAX2 depths are $\text{nc}(q_j) = 4, 4, 4, 4, 3, 3, 2$, so

$$\text{sum}_j 2^{-\{\text{nc}(q_j)\}} = 4/16 + 2/8 + 1/4 = 0.75 \quad 1$$

(checks **V1:E8**; **V2:B3**). Equation (6) is therefore not even an equality in the chapter’s own example.

We insist on separating two things that are easy to conflate:

- **Fig. 4 exhibits a misstatement.** The sum is 0.75, below 1. Step (7) needs only ≤ 1 , so the derivation goes through on this example unharmed (check **V2:B4**). Equations (5) and (8) both hold here too (checks **V2:B5**, **V2:B6**).
- **T1 exhibits the gap.** The sum is 1.25, above 1, and (7) fails.

Conflating them would overstate the finding. It also, we suspect, explains how the error survived: the one example in the paper fails the equation on the safe side, and its MAX subnetwork happens to be clean, so it cannot discriminate the literal and tree readings of nc either.

5. Adversarial verification: ten defences and their refutations

The audit above was subjected to an independent adversarial re-derivation whose brief was to *defend van Voorhis and refute the refutation*, working from the page images, implementing everything from scratch, and importing none of the first verifier’s code. Its verdict was **hole confirmed**. It could not break (8) or (12) either, with roughly 15,000 networks including a hill-climbing search aimed directly at them.

Ten steelman defences were attempted. We record all ten, because each is a route someone will otherwise re-walk.

#	defence	outcome
S1	“nc means branch-node depth, not literal comparator count.” Fig. 5 is captioned “ Symmetric MAX Subnetwork”, i.e. a redrawn tree.	Correct, and adopted — it is the only reading consistent with Fig. 5, Theorem 1’s proof and Table 1, and it makes (8) survive every test. It does not rescue (5) or (6): with tree nc, eq. (5) still returns $2+2 = 4 > 3 = p(2, T1)$, and nc(q _j) is unaffected, so eq. (6) is still 1.25.
S2	“eq. (6) only needs $\leq$ 1; the equality is a slip.”	Correct that only $\leq$ is needed, and this fully excuses Fig. 4’s 0.75 (§4.7). But T1 gives > 1 , the fatal side.
S3	“Read q _j in the pruned (N-1)-sorter, dropping comparators shared with the max path.”	Makes it worse: q ₀ shortens to 1, q ₂ stays 0, Kraft = 1.5.
S4	“Contract MAX2 to a tree and use tree depths for nc(q _j).”	Makes it worse: the q ₂ leaf is the root, at depth 0, and contraction cannot move it. Kraft ≥ 1.5 .
S5	“Exclude the root branch node from the q _j family.”	Then (7) loses the $2^{\{nc(c_{\{N-1\}})\}}$ term while (9) keeps it; the two sums no longer match and (8) does not follow. f would also no longer be Theorem 1’s f, so F(N) and Table 1 would be wrong.
S6	“There is a hidden standard-form / normalisation hypothesis.”	The chapter states none, and T1 is already in standard form (uncrossed, $a < b$, min to the low lead).

#	defence	outcome
S7	“There is a hidden minimality hypothesis.”	T1 is size-optimal: $ T1 = 3 = S(3)$. And eq. (1) defines $P(2,N) = \min_T p(2,T)$ over all N-sorters, so a single bad T suffices to break the per-network step.
S8	“ $p(2,T)$ might be a sum over the two paths, not a union.”	Then eq. (2) would be invalid, since the residual network has $ T - \text{union} $ comparators. p. 121’s “the greatest number of comparators that can be pruned” is unambiguous.
S9	“The failure only occurs in slack-rich, non-extremal networks, so it cannot threaten the bound.”	False , and this is the most consequential defence to have failed: 105 of 387 networks (27.1 %) have zero slack and a broken Kraft sum (§4.6).

#	defence	outcome
S10	The strongest: “MAX(T) and MAX2(T) are comparator-disjoint.” Sketch: every comparator of MAX(T) has both inputs on MAX-edges; q_j starts on c_j’s <i>lower</i> output, which is not a MAX-edge; a comparator with a non-MAX input is not in MAX(T); induct. Disjointness would make eq. (5) an exact union with nothing double-counted and force every q_j onto high leads throughout, hence prefix-free, hence Kraft — repairing (5) and (6) at one stroke.	Refuted. The premise “every comparator of MAX(T) has both inputs on MAX-edges” is exactly what pass-throughs violate. On T1, $MAX(T1) = \{c_0, c_1, c_2\}$ and $MAX2(T1) = \{c_1, c_2\}$ share two comparators, one of them the root branch node (check V2:C10). Across the sweep, MAX2 meets MAX in 233/387 (60.2 %) and meets a <i>branch node</i> of MAX in 232/387 (59.9 %) (check V2:D-1). Disjointness does hold on Batcher’s 8-sorter (check V2:C11) — again why Fig. 4 does not expose it.

S10 deserves emphasis. It is the argument a sympathetic reader reconstructs first; it is, we believe, the argument van Voorhis had in mind; and it fails for the same single reason as everything else — pass-through comparators, whose existence the N-1-comparator premise denies.

Localisation. The fault is exactly localised. Among the **151 clean** networks in the 387-network sweep, the number violating eq. (5) *as an upper bound on $p(2, T)$* , or eq. (6) *as the inequality ≤ 1* , or MAX/MAX2 disjointness, is **zero** (check **V2:D-2**); among the 236 with pass-throughs, eq. (6) breaks in 164. The chapter is correct about the networks it thinks it is describing. But 61.0 % of the sweep is not clean, so this is not a small residual case — it is the majority case.

We state the qualification precisely because it matters. Cleanliness does *not* make eq. (6) an equality: 5 of the 151 clean networks have Kraft sum ≤ 1 , and Batcher’s 8-sorter, which is clean, has 0.75. What cleanliness buys is the inequality ≤ 1 , which is all the derivation needs, and that is exactly the content of §7.5. Check **V2:D-2** tests the fatal directions only, and its label — “eq (5), eq (6) and MAX/MAX2 disjointness all hold” — overstates its own predicate; we flag this in NOTES.md.

Exhaustive attack. With no pruning at all: every sorting network on 3 channels with ≤ 4 comparators (42 of them) and every one on 4 channels with ≤ 6 comparators (912 of them) satisfies the conclusion $p(2, T) \geq \text{ceil}(\log_2 F(n))$ (checks **V2:E3**, **V2:E4**).

6. The literature record

We searched for any erratum, correction, gap-note, reproof or formalisation of the chapter’s equations (5)–(9): Google Scholar and Semantic Scholar citation lists, arXiv, DBLP, Knuth’s TAOCP Vol. 3 and its errata files, the Stanford InfoLab technical-report index, DTIC, Dobbelaere’s table together with its Wayback history, and Harder’s Isabelle sources.

1. Nothing exists. No erratum, no correction, no repair. The reason is stronger than “nobody found the bug”: **nobody has engaged with the argument at all.** Of roughly nineteen recorded citations of the chapter, the modern ones (e.g. Codish–Cruz-Filipe–Frank–Schneider-Kamp arXiv:1405.5754, arXiv:1507.01428, arXiv:1502.08008) all cite it for a *different* result — the one-value bound $S(n+1) \geq S(n) + \text{ceil}(\log_2 n)$, which is the content of the IEEE TC note, not of this chapter. A grep for `P(2,, MAX2, MAX subnetwork, Kraft` and `S(N-2)` across roughly twenty-five modern papers returns **zero hits**.

2. Knuth does not carry this bound. TAOCP Vol. 3, §5.3.4, exercise 42 (p. 240) is “(D. Van Voorhis.) Prove that $S(n) \geq S(n-1) + \text{ceil}(\lg n)$ ”, and the answer (p. 671) is Knuth’s own three-line proof citing **only** IEEE Trans. Computers C-21 (1972), 612–613. There is no two-value-pruning exercise, and Vol. 3 never cites the Plenum chapter. No entry in Knuth’s errata touches it. So TAOCP provides **no independent support** for $P(2, N)$.

3. Van Voorhis’s other works do not contain the argument. The IEEE TC note and its precursor Stanford technical report are the one-value bound only. The adjacent Stanford TR STAN-CS-71-238, “A Lower Bound for Sorting Networks that Use the Divide-Sort-Merge Strategy” (in `papers/`), bounds a *restricted class* of networks and concludes an $N (\log_2 N)^2$ rate; it is unrelated and is not the source of the two-value theorem.

4. No later restatement, reproof or formalisation. Harder (2020), the source of $S(11) = 35$ and $S(12) = 39$, does not cite the chapter; his Isabelle/HOL development contains zero occurrences of “Voorhis”. The Cruz-Filipe & Schneider-Kamp Coq formalisation covers generate-and-prune only. **There is no machine-checked version of the $P(2, N)$ bound.**

5. The provenance of $S(13) \geq 44$ is much weaker than generally assumed.

value	where it appears	basis
43	Wikipedia; Dobbelaere’s table before 2025-04-21	$S(12) + \text{ceil}(\log_2 13) = 39 + 4$
44	Dobbelaere’s table since 2025-04-21 only	$S(11) + P(2, 13) = 35 + 9$

The chapter’s own Table 1 (p. 128) gives $L(13) = 42$. The 44 entered the record via the changelog entry “2025-04-21 Tighter lower bounds for size, on suggestion of Jelmer Firet and based on principles in [VVoorh72]”; we found no publication or preprint by Firet.

$S(13) \geq 44$ has never appeared in a peer-reviewed publication. The best *published* lower bound is **43**. The 44 is a web-table entry sixteen months old, resting on a fifty-four-year-old argument that nobody has re-derived, re-proved or machine-checked — and whose two structural lemmas are false.

This substantially *lowers* the stakes of the finding: no peer-reviewed result is being overturned. It correspondingly *raises* the value of recording it, since the number is being propagated with no correct argument behind it.

7. The partial repair

We now prove equation (8) for clean sorters. The proof is elementary; its content is a two-colouring of leads that supplies the antichain step van Voorhis assumed, and that identifies pass-throughs as the sole obstruction.

Throughout, T is an N -sorter, $N \geq 3$, with branch tree B , and $a(c)$, $f(B)$, $W(x, y)$, $p(2, T)$ are as in §2.

7.1 The exact decomposition

Fact 1 (first meeting). In the scenario (x, y) the max and the second max first meet at $LCA_B(x, y)$, the least common ancestor of leaves x and y in the branch tree.

Proof. Until they meet, the second max is larger than every value on its channel other than the max, so it takes the high output at every comparator it enters; that is, it follows y ’s max-path. The max likewise follows x ’s max-path. Let d be the first comparator common to p_x and p_y . Before d the two paths share no comparator, and they start on distinct input leads, so they occupy distinct leads throughout; hence they arrive at d on its two different input leads, and d is a branch node. Being the first common comparator, it is the first common branch node, which is $LCA_B(x, y)$.

Fact 2 (decomposition). With $c = \text{LCA}_B(x, y)$,

$$|W(x, y)| = (x) + e(y, c) + r(c),$$

where (x) is the number of comparators on x 's max-path, $e(y, c)$ the number strictly before c on y 's max-path, and $r(c)$ the number of comparators the second max traverses after c that the max does **not**. Moreover $r(c)$ depends only on c .

Proof. After c the max is still the global maximum, so it continues along p_x ; its total contribution is (x) . The second max contributes the $e(y, c)$ comparators strictly before c , then c itself (already counted), then those after c not shared with the max, of which there are $r(c)$ by definition. Immediately after c the max occupies c 's high output lead and the second max c 's low output lead; that state, and hence the whole subsequent evolution of both values, is determined by c and the network alone, not by x or y .

Also $(x) = e(x, c) + g(c)$ where $g(c)$ counts the comparators from c to o_N inclusive on the max's onward path.

Comparing with the chapter: van Voorhis wrote $|W| = nc(c) + nc(q_c)$, which double-counts the $ov(c) := nc(q_c) - r(c)$ comparators shared after c . That is precisely where eq. (5) goes wrong. Fact 2 is verified by path contraction on every network tested (checks **V1:B4**, **V1:B5**).

7.2 Red and blue leads

Definition. A lead is **red** if some one-hot max scenario puts the maximum on it, and **blue** otherwise.

(R1) Every low output lead is blue.

Proof. The maximum is strictly greater than the other input of any comparator it enters, so it leaves on the high output. It therefore never occupies a low output lead.

(R2) A high output lead is red iff its comparator has at least one red input lead.

Proof. () If an input lead is red, then in that scenario the max enters the comparator and exits on the high output. () If the high output lead is red, the max occupies it, and it can only have arrived by traversing the comparator, so it occupied one of the input leads, which is therefore red.

All N input leads of the network are red, since the max-path from channel k begins on input lead k .

(R3) A comparator is a branch node iff both its input leads are red; it is a pass-through iff exactly one is red; and it lies outside $\text{MAX}(T)$ iff neither is.

Proof. A max-path arrives at a comparator on lead ℓ exactly when ℓ is red and is an input lead of that comparator. Branch node means max-paths arrive on both leads; membership in $\text{MAX}(\mathbf{T})$ means at least one.

(R1)–(R3) are machine-checked with zero violations over 900 constructed sorters (check **V1:G1**).

7.3 Blue trapping

Lemma 1 (blue trapping). Let $\mathbf{T}$ be clean. In the scenario $(\mathbf{x}, \mathbf{y})$ with first meeting at $\mathbf{c}$, the second max occupies only **blue** leads after $\mathbf{c}$.

Proof. By induction along the second max’s trajectory. Immediately after $\mathbf{c}$ it is on $\mathbf{c}$ ’s low output lead, blue by (R1). Suppose it is on a blue lead ℓ and enters comparator $\mathbf{d}$ on ℓ . Since ℓ is blue, $\mathbf{d}$ does not have two red input leads, so by (R3) $\mathbf{d}$ is not a branch node; since $\mathbf{T}$ is clean, $\mathbf{d}$ is not traversed by any max-path at all. Then by (R3) $\mathbf{d}$ has *no* red input lead. Hence $\mathbf{d}$ ’s low output is blue by (R1) and $\mathbf{d}$ ’s high output is blue by (R2), and the second max leaves $\mathbf{d}$ on a blue lead whichever it takes.

(Check **V1:G2**: over the clean networks in a 900-network population, 128 of 128, zero violations.)

Corollary 2 (no re-meeting). In a clean $\mathbf{T}$ the two values never meet again after $\mathbf{c}$. Hence $\text{ov}(\mathbf{c}) = 0$, $\mathbf{r}(\mathbf{c}) = \mathbf{nc}(\mathbf{q}_{\mathbf{c}})$, and equation (5) is exact for the pairs it is applied to.

Proof. Suppose both enter comparator $\mathbf{d}$ after $\mathbf{c}$. Then the max occupies an input lead of $\mathbf{d}$, so that lead is red and $\mathbf{d} \in \text{MAX}(\mathbf{T})$; by cleanliness $\mathbf{d}$ is a branch node, so by (R3) *both* its input leads are red. But the second max arrives at $\mathbf{d}$ on a blue lead by Lemma 1 — contradiction.

Corollary 3. In a clean $\mathbf{T}$, after $\mathbf{c}$ the second max takes the **high** output lead of every comparator it enters; and $\mathbf{nc}(\mathbf{c}) = \mathbf{a}(\mathbf{c})$ for every branch node $\mathbf{c}$, literal counts coinciding with branch-tree quantities.

Proof. By the proof of Lemma 1, any comparator $\mathbf{d}$ entered after $\mathbf{c}$ is outside $\text{MAX}(\mathbf{T})$; by Corollary 2 the max is not at $\mathbf{d}$; and the second max is strictly larger than every value other than the max, so it is the larger input at $\mathbf{d}$ and leaves high. For the second statement: in a clean network every traversed comparator is a branch node, so counting comparators along a max-path is the same as counting branch-tree edges, and $\mathbf{nc}(\mathbf{c}) = \mathbf{lp}(\mathbf{L}(\mathbf{c})) + \mathbf{lp}(\mathbf{R}(\mathbf{c})) + \mathbf{depth_B}(\mathbf{c}) + 1 = \mathbf{a}(\mathbf{c})$.

7.4 The MAX2 in-tree and the antichain

Lemma 4 (the tree). Let $\mathbf{T}$ be clean. For a lead ℓ occupied by a second max after separation, let $\mathbf{succ}(\ell)$ be the high output lead of the next comparator touching ℓ (and let ℓ be the root if no

comparator follows). By Lemma 1 and Corollary 3 the second max's motion after c is exactly iteration of **succ**, **independently of the scenario**. The set of such leads, under **succ**, is a binary in-tree rooted at the output lead of o_{N-1} .

Proof. **succ** is a function of the lead alone, so the trajectory from $_c$ is determined by c ; this is Fact 2's " $r(c)$ depends only on c " made explicit at the lead level. Every trajectory terminates at o_{N-1} , since T sorts and the second largest input value must emerge there. A lead m has at most two **succ**-preimages, namely the two input leads of the comparator whose high output is m . So the reachable structure is an in-tree with branching at most 2, rooted at o_{N-1} ; the depth of $_c$ is the number of comparators the second max traverses from $_c$ to o_{N-1} .

Lemma 5 (the antichain — the step the chapter omits).

Let T be clean. The $N-1$ sources $_c$ — the low output leads of the branch nodes c — are pairwise non-ancestral in that in-tree.

Proof. By Corollary 3, every lead strictly after $_j$ on its **succ**-path is a **high** output lead. Every $_k$ is a **low** output lead, and $_j \prec _k$ for $j \neq k$ since distinct branch nodes have distinct low output leads. So no $_k$ lies on the path from $_j$.

7.5 The clean-case theorem

Theorem 6 (equation (8) for clean sorters). Let T be a clean N -sorter, $N \geq 3$, with MAX branch tree B . Then

$$p(2, T) \geq \lceil \log_2 f(B) \rceil, \quad \text{and hence} \quad |T| \geq S(N-2) + \lceil \log_2 f(B) \rceil$$

Proof. For each branch node c let x_c be a deepest leaf of $L(c)$ and y_c a deepest leaf of $R(c)$; then $\text{LCA}_B(x_c, y_c) = c$.

Step 1: $|W(x_c, y_c)| = a(c) + nc(q_c)$. By Fact 2 and Corollary 2, $|W(x_c, y_c)| = (x_c) + e(y_c, c) + nc(q_c)$ with $(x_c) = e(x_c, c) + g(c)$. Since T is clean, every comparator on a max-path is a branch node, so $e(x_c, c) = lp(L(c))$, $e(y_c, c) = lp(R(c))$ (the leaves chosen are deepest), and $g(c) = \text{depth}_B(c) + 1$. Adding, $(x_c) + e(y_c, c) = lp(L(c)) + lp(R(c)) + \text{depth}_B(c) + 1 = a(c)$.

Step 2: the per-node inequality. $p(2, T) = \max_{x, y} |W(x, y)| \geq |W(x_c, y_c)|$, so $nc(q_c) \leq p(2, T) - a(c)$ for every branch node c .

Step 3: Kraft. By Lemmas 4 and 5 the $N-1$ leads $_c$ form an antichain at depths $nc(q_c)$ in a binary in-tree, so Kraft's inequality gives $\sum_c 2^{-nc(q_c)} \leq 1$.

Step 4: combine.

$$1 \geq \sum_c 2^{-nc(q_c)} \geq \sum_c 2^{-(p(2, T) - a(c))} = 2^{-p(2, T)} f(B),$$

so $2^{\{p(2,T)\}} \geq f(B)$; since $p(2,T)$ is an integer, $p(2,T) \geq \lceil \log_2 f(B) \rceil$. The second statement follows from the pruning decomposition $|T| = |\text{residual}| + |W|$ with the residual an $(N-2)$ -sorter (check **V1:B5**).

Two things changed relative to the chapter, and both are essential:

- **Equation (6) becomes an inequality, not an equality.** That is exactly why Batchier’s own 8-sorter gives 0.75 and not 1 (§4.7), and the argument is unharmed.
- **The antichain is proved, not assumed.** Lemma 5 is the sentence the chapter never wrote, and Lemma 1 is what makes it available.

Machine support: check **V1:G1** verifies (R1)–(R3) and the pass-through characterisation; **V1:G2** verifies Lemma 1’s conclusion on all 128 clean networks of a 900-network population; **V1:G3** verifies $\sum_c 2^{\{-nc(q_c)\}} \leq 1$ on the same 128; **V1:G4** verifies the conclusion of Theorem 6 on all 900. We stress that Theorem 6 is a human proof: the checks test its lemmas and conclusion on samples, they do not formalise it. See §11.

7.6 What is closed off: three refuted repair routes

Theorem 6 covers the clean case. Cleanliness is not a small hypothesis — 61 % of the audited population fails it — so the general case remains. We record what does *not* work, with counterexamples, so that the routes are not re-walked.

7.6.1 Per-node charging is dead For a branch node c put $W^*(c) := \max\{|W(x,y)| : \text{LCA}_B(x,y) = c\}$, the best pruning that separates at c . (The second verifier writes u_j for the same quantity and calls the statement below (K); the repair analysis writes W^* and calls it LEMMA . They are the same statement.)

LEMMA (slack-Kraft). For every N -sorter T , $\sum_{\text{c branch node}} 2^{\{a(c) - W^*(c)\}} \leq 1$.

LEMMA implies equation (8). Since $W^*(c) \leq p(2,T)$,

$$\begin{aligned} f(B) &= \sum_c 2^{\{a(c)\}} = \sum_c 2^{\{W^*(c)\}} \cdot 2^{\{a(c)-W^*(c)\}} \\ &\leq 2^{\{p(2,T)\}} \cdot \sum_c 2^{\{a(c)-W^*(c)\}} \leq 2^{\{p(2,T)\}}. \end{aligned}$$

LEMMA holds for clean sorters — that is Theorem 6’s proof, since $W^*(c) \geq |W(x_c, y_c)| = a(c) + nc(q_c)$. It rescues the disputed 3-sorter exactly: there $W^* = (3,3)$, $a = (2,2)$, and the sum is $1/2 + 1/2 = 1$ (check the T1 line of `verify_kraft_dispute.py`’s Part D’ — printed, not asserted).

LEMMA is false. Adversarial hill-climbing produced the 6-sorter

$$\begin{aligned} T_3 = & [(2,3), (1,2), (0,4), (3,5), (2,4), (1,2), (4,5), (2,3), \\ & (0,3), (3,4), (1,3), (1,2), (0,3), (0,1), (1,2)] \end{aligned}$$

with 15 comparators ($S(6) = 12$) and exactly one pass-through. Its branch-node exponents are $a = \{3,3,3,3,5\}$ and, **taking the**

maximum over all admissible pairs at each node, $W^* = \{6, 5, 6, 8, 6\}$, so

$$\text{sum_c } 2^{\{a(c)-W^*(c)\}} = 1/8 + 1/4 + 1/8 + 1/32 + 1/2 = 33/32 = 1.03125 > 1.$$

Checks **V1:G5** (re-derived from scratch) and **V2:D'3**.

Note precisely what does and does not follow.

- **Equation (8) is not refuted.** On T3, $f(B) = 64$ and $p(2, T3) = 8$, so $\text{ceil}(\log_2 64) = 6 \leq 8$ comfortably (checks **V1:G6**, **V2:D'5**). LEMMA was only ever *sufficient* for (8), never necessary: it compares each $a(c)$ against that node's own best pruning $W^*(c)$, which can be far below the global maximum $p(2, T)$.
- **Every per-node charging repair is now closed off**, because the strongest member of the family — best-choice W^* — is false. Any valid proof of (8) must be **global**: it must exploit the fact that the branch nodes cannot all have small slack simultaneously, rather than bounding term by term.
- **Sampling was not evidence.** 900 randomly constructed sorters produced zero LEMMA violations with supremum exactly 1.000000 (check **V1:G6b**, retained deliberately as a caution). An adversarial search found a counterexample immediately.

7.6.2 The pairing rule is load-bearing Replacing $W^*(c)$ by $|W(x_c, y_c)|$ — van Voorhis's own deepest-leaf pairing — gives a *stronger* statement, and it is false for a smaller reason. Check **V1:G7** exhibits a constructed 7-channel sorter with 19 comparators,

$[(5, 6), (2, 5), (1, 5), (5, 6), (1, 2), (3, 4), (4, 6), (0, 2), (2, 6), (0, 1), (2, 3), (3, 4), (4, 5), (1, 2), (2, 3), (3, 4), (0, 1), (2, 3), (1, 2)]$

whose deepest-leaf slack-Kraft sum is $1.078125 > 1$ while its best-choice sum is 0.671875 . (Unlike T3 this network is not pinned in the source: it is found at run time by a seeded search over $n = \{5, 6, 7\}$, so it is reproducible but would change if the seed changed.) So any repair must take the maximum over admissible pairs at each branch node; the chapter's habit of fixing "the longest path through $L(c_j)$ " is not merely a notational convenience. (On T3 the two sums coincide at 1.03125, which is why T3 kills the whole family and not just the deepest-leaf version.)

7.6.3 Other routes tried, and why they are short

route	outcome
Kraft on $\text{MAX}(T/x)$ for a fixed channel x	Sound and unconditional, but it only “sees” the $\text{lp}(B)$ ancestors of x , not all $N-1$ branch nodes. Yields $p(2,T) \geq \text{lp}(B) + \text{ceil}(\log_2(N-1))$, which at $n = 13$ gives 43 — the one-value bound again.
Summing that over all x (pair counting)	Gives the clean $\text{sum}_{\{x,y\}} 2^{- W(x,y) } \leq 1$, but the per-node consequence is short by a factor $2^{\text{depth}(c)}$: it yields $N(N-1) = 156$ at $n = 13$ where $f = 392$ is needed.
Dropping non-antichain sources from van Voorhis’s sum	The surviving mass collapses ($2^{\text{a-ov}}$ on a strict subset); on T_1 it gives only $p(2,T) \geq 1$.
Eliminating pass-throughs by rewriting T	Impossible in general: the optimal 3-sorter T_1 has one and none of its comparators is redundant.
The union-based repair (K) (= LEMMA)	Refuted, §7.6.1.
MAX/MAX2 comparator-disjointness	Refuted, defence S10 in §5.

Without the Kraft step the argument yields only $p(2,T) \geq \max_j n(c_j)$, which for the f -minimising 13-leaf trees is $35 + 7 = 42$. **The Kraft step carries the entire 44, and the Kraft step is the broken one.**

(Caveat: the four rows above are analytic assessments recorded in the repair analysis. Unlike the rest of this paper they are not each tied to an individual check ID; the arithmetic is elementary but a referee is entitled to ask for it in full. Flagged in NOTES.md.)

8. Status, evidence, and what a full repair must achieve

8.1 The open statement

Conjecture (the two-value bound). For every N -sorter T with MAX branch tree B ,

$$p(2,T) \geq \text{ceil}(\log_2 f(B)).$$

Equivalently, $|T| \geq S(N-2) + \text{ceil}(\log_2 f(\text{MAX}(T)))$, and consequently $P(2,N) \geq \text{ceil}(\log_2 F(N))$.

Known. True for clean T (Theorem 6). True on every network in the artifact. Consistent with every exact value of $S(n)$. **Not proved in general, and not refuted.**

8.2 Empirical evidence

population	source	networks	violations of the conjecture
structured families, $n = 3 \dots 12$ (bubble, Batcher, odd-even transposition, insertion-extension, thinned random prefixes)	verifier 1, Part B	43	0 (check V1:B6)
randomised sorters, $n = 4 \dots 10$	verifier 1, Part F	560	0 (checks V1:F1 , V1:F2)
randomised sorters, $n = 3 \dots 12$	verifier 1, Part G	900	0 (check V1:G4)
structured + randomised, $n = 3 \dots 10$	verifier 2, Part D	387	0 (check V2:D3)
exhaustive: every 3-channel comparator sequence of length ≤ 4 that sorts	verifier 2, Part E	42	0 (check V2:E3)
exhaustive: every 4-channel comparator sequence of length ≤ 6 that sorts	verifier 2, Part E	912	0 (check V2:E4)
adversarial hill-climb targeting (8) and (12) directly	scratch, <code>.build/v3-theory-audit/</code>	$\sim 14,500$	0

The four main sweeps of the two shipped verifiers exercise **1,890** constructed

sorters ($43 + 560 + 900 + 387$); auxiliary checks add roughly 60 more (the G7 search, the seven `realize_shape` networks of check **V1:G8**, the Batcher family of check **V1:H2**, and the fixed objects T1, T2, T3 and Batcher-8), and the two exhaustive enumerations add 954. The enumerations are of *networks*, not of distinct minimal sorters: no symmetry reduction or redundancy filtering is applied, so sequences containing redundant comparators are included.

Two further sweeps live in scratch directories (`.build/v3-theory/stress.py`, roughly 5,000 additional networks across five seeds; the adversarial hill-climb above) and are *not* part of either script’s exit status. The prose reports produced during the work quote aggregate figures of “~5,600” and “~6,500” constructed sorters, which fold those scratch sweeps in. We do not: the numbers above are what the two shipped scripts exercise.

A second, independent consistency test: the conjecture implies $\text{ceil}(\log_2 F(N)) \leq S(N) - S(N-2)$ for every N where both are known. Over $N = 3..12$ the slacks are

0, 0, 0, 1, 0, 0, 1, 2, 2, 1

(check **V1:A6**). Four of the ten are zero, i.e. the bound is exactly tight; a single negative entry would have refuted the theorem outright. Similarly $\text{ceil}(\log_2 N) \leq S(N) - S(N-1)$ for $N = 2..12$ (check **V1:A7**) and the chapter’s $P(2,11) = 9 \leq S(11) - S(9) = 10$ (check **V1:A8**).

We are explicit about the epistemic weight of this: it is consistency evidence, not proof, and §7.6.1 is a cautionary tale about exactly this kind of evidence — 900 samples with supremum exactly 1 preceded an immediate adversarial counterexample to LEMMA .

8.3 What a full repair must achieve

1. **It must be global.** Per-node charging is refuted (§7.6.1). A proof must use the impossibility of all branch nodes having small slack simultaneously.
2. **It must handle pass-throughs, not exclude them.** They are the majority case (61.0 %), they occur in size-optimal networks (T1 is optimal), and they cannot be rewritten away.
3. **It must survive the tight regime.** 27.1 % of the audited population has zero slack in the conclusion *and* a broken Kraft sum; a repair that only works where there is slack proves nothing new.
4. **It must reach $\text{ceil}(\log_2 f(B))$, not merely $\max_j \text{nc}(c_j)$.** The sound unconditional fragments give 42 and 43 at $n = 13$; the target is 44.
5. **Or it must be replaced.** A different route to $P(2,13) \geq 9$ would serve equally well. Nothing in this paper suggests one.

Failing all of that, the honest state of the record is $43 \leq S(13) \leq 45$, with 44 an unrefuted conjecture.

8.4 Three adjacent results, recorded because they are cheap and correct

(a) **The MIN dual.** The chapter observes (p. 127) that the argument “works equally well to prune paths followed by the smallest two input values to o_1 and o_2 ”. Defining $\text{MIN}(T)$ dually — the union of the paths followed by the smallest value from each input to o_1 — we obtain, *conditional on the same equation (8)*,

$$|T| \geq S(N-2) + \max(\text{ceil}(\log_2 f(\text{MAX}(T))), \text{ceil}(\log_2 f(\text{MIN}(T)))) .$$

Zero violations in the artifact (checks **V1:C1**, **V1:F3**: 560/560). At $n = 13$ this doubles the structural constraint for free: a 44-comparator 13-sorter must have **both** $f(\text{MAX}) \leq 512$ and $f(\text{MIN}) \leq 512$, i.e. both trees must lie in the same admissible set of 84 plane / 6 abstract shapes, since the admissible set is a property of f and does not depend on which tree it is applied to. (*The artifact registers a check D7 for this last sentence, but its predicate is the literal **True**; it asserts nothing and we do not cite it as evidence. Flagged in NOTES.md.*)

(b) **$P(2,11) = 9$ reduces to a single shape.** $F(11) = 256 = 2^8$ exactly, so eq. (12) yields only $P(2,11) \geq 8$ (check **V1:A4**). 256 is attained by **exactly one** abstract shape (8 plane shapes) — root split 4|7, height 4, leaf depths 3,3,3,3,3,4,4,4,4,4,4 — and the next attainable value is 272, with $\text{ceil}(\log_2 272) = 9$ (check **V1:H1**). So van Voorhis’s unproved “it turns out that” reduces to: *every 11-sorter whose MAX branch tree is that one shape admits a pruning of at least 9 comparators*. The claim is not vacuous, because that shape is realizable (below), and it is consistent, since $9 \leq S(11) - S(9) = 10$ (check **V1:A8**). Reconstructing this argument is self-validating — the answer is known — and it is the *only* known technique for beating $\text{ceil}(\log_2 F(N))$.

(c) **Every admissible shape is realizable, cleanly.** Every binary tree shape S with n leaves is the MAX branch tree of some **clean** n -sorter: run the tournament S describes (each comparator on the two subtree-winner channels, winner always to the higher channel, so the overall winner lands on channel $n-1$), then sort channels $0..n-2$. Each tournament comparator has two red inputs, hence is a branch node by (R3); the appended sorter never touches channel $n-1$, so no max-path is extended. Machine-verified for the unique $n = 11$ minimiser and all six admissible $n = 13$ shapes (check **V1:G8**). Two consequences: $P(2,11) = 9$ cannot be dodged by unrealizability, and a purely shape-level MAX/MIN incompatibility at $n = 13$ is impossible — Batcher’s n -sorter has $f(\text{MAX}) = f(\text{MIN}) = F(n)$ for $n = 3, 4, 6, 7, 8, 12$ (check **V1:H2**), and a constructed 13-sorter has $f(\text{MAX}) = 512$ with $f(\text{MIN}) = 392$, both admissible (check **V1:H3**). Only a *size-conditioned* MAX/MIN argument could survive, and it would have to consume the comparator budget rather than reason about shapes.

(d) **A note on the generalisation.** The two-value bound generalises to unequal residual bounds: if b_j is any lower bound on the size of the $(N-2)$ -sorter

left by pruning at branch node c_j , then $|T| \geq \text{ceil}(\log_2 \sum_j 2^{\{b_j + \text{nc}(c_j)\}})$. This coincides with the **1+max**-Huffman kernel used in Harder’s Theorem 26 — the identity $\text{max_plus_1_huffman}(b_1..b_m) = \text{ceil}(\log_2 \sum_i 2^{\{b_i\}})$ is verified exhaustively for $k \leq 5$, $b \leq 6$ plus 40,000 random multisets (check **V1:D1**). Its proof uses equations (5) and (6) and nothing else beyond the verified decomposition, so it is **exactly as sound as the published 44 and no more**. With uniform $b_j = S(11) = 35$ it reproduces 44 and yields nothing further, and a uniform +1 would kill every admissible shape but asserts $S(11) \geq 36$, which is false (checks **V1:D3**, **V1:D6**; note that as written these two evaluate only the first of the six admissible shapes — the statement does hold for all six, but the checks do not establish the quantifier, and we flag this in **NOTES.md**). We record the generalisation only to note that repairing (5)/(6) would repair it too.

9. Reproducibility

9.1 The artifact

file	sha256	checks	role
tools/verify_huffman2.py	802c6cbdccc4110fb228c255b5a65688afcd11e7e0f08b062f694a3e4	44	Table 1 reproduction, the counterexample, prevalence, the clean-case repair, the MIN side.
tools/verify_kraft76des5820e3fe633336ac47023ac09e6164ed199c11933ac3db0ecd1567442de		36	adversarial re-derivation, written from the page images, importing none of the first script’s code.

Both are pure Python standard library, read-only, no network access, fixed seeds, no environment sensitivity.

9.2 Exact commands and observed output

Run from the repository root. Python 3.14.4 on macOS (Darwin 25.5.0, Apple silicon).

```
$ python3 tools/verify_huffman2.py
```

```

...
    all 48 checks PASSED.
exit status 0          wall 45.98 s          (user 44.56 s, sys 0.16 s)

$ python3 tools/verify_kraft_dispute.py
...
VERDICT: (A) HOLE CONFIRMED.  eq (5) and eq (6) are false under
every reading of the chapter's definitions; eq (7) is derived from
them and from nothing else; therefore eq (8), (10) and (12) --
and hence  $P(2,13) \geq 9$  and  $S(13) \geq 44$  -- are not proved by this
chapter.  They are also not refuted.
exit status 0          wall 1.87 s          (user 0.93 s, sys 0.03 s)

Additional flags: verify_huffman2.py accepts --quiet (verdicts only) and
--fast; verify_kraft_dispute.py accepts --fast and --seed SEED.

```

--fast does not reproduce this paper. It changes sample sizes, not the number of checks (48 and 36 either way), and it shrinks them substantially: the first verifier's Part B sweep drops $43 \rightarrow 33$, Part F $560 \rightarrow 150$ and Part G $900 \rightarrow 320$; the second verifier's population drops $387 \rightarrow 93$ and its $n = 4$ exhaustive enumeration drops from 912 networks (length ≤ 6) to 12 (length ≤ 5). Every headline percentage in §4.6 and every population count in §8.2 requires the full runs above. **--fast** is a smoke test.

The default seed of `verify_kraft_dispute.py` is 20260818; changing **--seed** changes the composition of the 387-network population and therefore every percentage in §4.6. The first verifier's seeds are fixed in source (20260818 for Parts B and F, 4242 for Part G, 1 for the check **V1:G7** search, 11 for check **V1:D1**) and are not exposed as flags.

Selected verbatim output lines, quoted because the paper's numbers are exactly these:

```

F(N), N=1..16 : [0, 2, 8, 16, 36, 52, 80, 96, 168, 200, 256, 288, 392, 424, 480, 512]
L(N), N=1..16 : [0, 1, 3, 5, 9, 12, 16, 19, 24, 28, 33, 37, 42, 46, 51, 55]
NB L(11)=33 was 1972's best; S(11)=35 is Harder 2020. Feeding the
    modern value into eq (2) with N=13 gives  $35 + 9 = 44$ .
slack S(N)-S(N-2)-ceil(log2 F(N)), N=3..12 : [0, 0, 0, 1, 0, 0, 1, 2, 2, 1]

T = batcher(3) = [(0, 1), (0, 2), (1, 2)], |T| = 3 = S(3), sorts = True
comparators traced by MAX paths: [0, 1, 2]  branch nodes: [0, 2]
nc from the branch tree: {2: 2, 0: 2} -> f = 8
nc as eq (9) DEFINES it: {0: 2, 2: 3} -> f = 12
p(2,T) by brute force over all ordered pairs = 3

560 randomly constructed sorters, n = [4, 5, 6, 7, 8, 9, 10]
eq (6) Kraft violated in 192 (34%); eq (9) literal violated in 90 (16%)

```

900 constructed sorters ($n = [3, 4, 5, 6, 7, 8, 9, 10, 11, 12]$); pass-through-free: 128
 $|T3| = 15$ ($S(6) = 12$), pass-throughs = 1, $f = 64$, $p(2, T3) = 8$
best-choice sum = 1.03125, deepest-leaf sum = 1.03125

networks audited: 387 (all verified sorters by the 0/1 principle)
MAX(T) has ≥ 1 pass-through comparator 236 61.0%
eq (5) RHS strictly EXCEEDS $p(2, T)$ (fatal) ... 172 44.4%
eq (6) sum > 1 (fatal direction) 164 42.4%
eq (8) fails, literal nc 102 26.4%
eq (8) fails, TREE nc 0 0.0%
CONCLUSION $p(2, T) \geq \text{ceil}(\log_2 F(n))$ fails ... 0 0.0%
 $p(2, T) = \text{ceil}(\log_2 f_{\text{tree}})$ exactly, i.e. NO slack 220 56.8%
... of those, eq (6) also fails with sum > 1 105 27.1% of all
networks whose MAX(T) is pass-through-FREE ... 151 39.0%
... of which any of eq(5)/eq(6)/disjointness fails: 0

9.3 Claim-to-check index

Check IDs collide between the two scripts — both define an A1, a B1, a C1, a D1, an E3 and so on, meaning different things. Throughout this paper, **V1:** prefixes an ID in `tools/verify_huffman2.py` and **V2:** an ID in `tools/verify_kraft_dispute.py`.

§	claim	check(s)
2	$f = \text{Theorem 1's}$ recursion; $F(N)$ column reproduces Table 1 for N ≤ 16	V1:A1, V1:A2; V2:A1
2	branch tree always has n leaves and $n-1$ nodes; $nc_{\text{actual}} \geq nc_{\text{tree}}$	V1:B1
2	pruning the two paths leaves an $(N-2)$ -sorter	V1:B5
3	$F(13) = 392$, $\text{ceil}(\log_2 392) = 9$	V2:A2
3	the chapter's $L(N)$ column reproduces from $P(1, \cdot)$, $P(2, \cdot)$	V1:A5 (but see §9.5 item 4)
4.1	$T1$ sorts and is size-optimal	V1:E0, V2:C0
4.2	$\text{MAX}(T1)$ has 3 comparators, not 2; branch nodes still 2	V1:E1, V2:C1, V2:C2

§	claim	check(s)
4.3	eq. (5) is false and exceeds $p(2, T1)$	V1:E2, V1:E3, V1:B4b, V2:C3, V2:C4
4.4	eq. (6) Kraft sum = $1.25 > 1$	V1:E4, V1:B2, V1:B3, V2:C5
4.5	eq. (8) false under the literal reading of (9), true under the tree reading	V1:E5, V1:E6, V2:C6, V2:C7, V2:C8
4.6	prevalence (existence and universality only — see §4.6, §9.5 item 5)	V2:D0, V2:D1, V2:D2, V2:D3; V1:F4
4.7	Fig. 4/5: nc multiset $\{3, 3, 3, 3, 4, 4, 5\}$, $f = 96 = F(8)$, Kraft = 0.75 but harmless	V1:E7, V1:E8; V2:B1–V2:B6
5	S10 (disjointness) refuted; holds on Batchers-8	V2:C10, V2:C11, V2:D-1
5	pass-throughs are the sole obstruction (151 clean, 0 failures in the fatal directions)	V2:D-2
5	exhaustive $n = 3$ (42 networks), $n = 4$ (912 networks)	V2:E3, V2:E4
7.1	Fact 2, the exact decomposition	V1:B4, V1:B5
7.2	(R1)–(R3) red/blue structure and the pass-through characterisation	V1:G1
7.3	Lemma 1 (blue trapping) on clean networks, 128/128	V1:G2
7.4–7.5	Kraft ≤ 1 on clean networks; Theorem 6's conclusion on all 900	V1:G3, V1:G4
7.6.1	LEMMA / (K) refuted by T3; eq. (8) still holds on T3	V1:G5, V1:G6, V2:D'3, V2:D'5
7.6.1	sampling caution: 900 samples, 0 violations, supremum 1.000000	V1:G6b

§	claim	check(s)
7.6.2	deepest-leaf pairing insufficient (7-channel, 19 comparators)	V1:G7
8.2	conjecture unviolated across all populations	V1:B6, V1:F1, V1:F2, V1:G4, V2:D3, V2:E3, V2:E4
8.2	consistency with exact $S(n)$	V1:A6, V1:A7, V1:A8
8.4(a)	MIN dual unviolated	V1:C1, V1:F3 (V1:D7 is vacuous — see §9.5 item 1)
8.4(b)	$F(11) = 256$, unique minimising shape, next value 272	V1:A4, V1:H1
8.4(c)	realizability of all admissible shapes, cleanly	V1:G8
8.4(c)	shape-only MAX/MIN incompatibility refuted	V1:H2, V1:H3
8.4(d)	Huffman identity; uniform-leaf collapse to 44	V1:D1, V1:D2, V1:D3, V1:D4, V1:D5, V1:D6 (D3/D6: see §9.5 item 2)

9.4 Hygiene

Every network the artifact touches is **constructed** — Batcher odd-even merge-sort, bubble, insertion, odd-even transposition, insertion-extension, the tournament construction of §8.4(c), random comparator prefixes in front of a bubble network followed by randomised redundant-comparator removal, and random channel relabellings retained only when they still sort — or brute-force enumerated. No witness network for any open case is embedded anywhere.

Four comparator lists appear as literals in the source, and we name them so that no reader has to hunt: $T1 = [(0,1), (0,2), (1,2)]$ and $T2 = [(0,1), (1,2), (0,1)]$, the two optimal 3-sorters used as counterexample and control; $T3$, the 15-comparator 6-sorter of §7.6.1, which was found by an adversarial hill-climb in a scratch directory and is *pinned* in both scripts as a fixed test object; and the transcribed $F(N)$ row of the chapter’s Table 1, which is used as an assertion *target* (the script’s own F is computed by DP). Each of the three networks is re-verified from scratch on every run — sorting by the 0/1 principle, branch tree, $a(c)$, $W^*(c)$ and $p(2, T)$ all recomputed — but $T3$ is not re-derived; reproducing its discovery requires the scratch search. $T3$ has 15

comparators on 6 channels, well above $S(6) = 12$, and is a witness for nothing.

No comparator count is used anywhere in the artifact as a search target, bound, feature or stopping condition; the number 44 appears only as the tabulated bound under audit.

9.5 Known weaknesses of the artifact

We list these because a reader checking our claims against the code will find them, and because two of them affect what the checks can be cited for.

1. **D7 asserts nothing.** Its predicate is the literal `True`. It inflates the first verifier’s check count by one. We do not cite it (§8.4(a)).
2. **D3 and D6 evaluate one admissible shape, not all six.** Both index the first element of the admissible list. The universal statements are true — we confirmed them independently for all six shapes — but the checks as written do not establish them (§8.4(d)).
3. **D-2’s label overstates its predicate.** It tests eq. (5) only in the fatal direction and eq. (6) only as > 1 ; eq. (6) as an *equality* does fail on 5 of the 151 clean networks (§5).
4. **Check A5 is not a pure derivation.** The chapter’s $L(N)$ column is regenerated from $P(1,N)$ and $P(2,N)$, but $P(2,11)$ is hand-set to 9 — the chapter’s own MIN-dual result, which check A4 has just shown does *not* follow from eq. (12). The patch is commented in the source. A5 therefore verifies that the chapter’s table is internally consistent *given* its unproved $P(2,11) = 9$, not that the column derives from proved inputs.
5. **Several headline statistics are printed, not asserted** (§4.6).
6. **G6b’s label and predicate point in opposite directions.** The label correctly warns that 900 clean samples are not evidence for LEMMA ; the predicate fails if any sample violates it. This is deliberate (it is a regression guard) but reads oddly.
7. **The two scripts quantify $p(2,T)$ differently** — the first over ordered pairs, the second over unordered pairs. The quantity is symmetric in its two arguments (the union $W(x,y)$ is unchanged by swapping which channel carries the maximum), so the two agree; but the agreement is a small theorem, not an identity of code.
8. **Both scripts’ module documentation has drifted.** The first verifier’s header enumerates Parts A, B, C, D, F only, while Parts E, G and H are run and contribute 21 of the 48 checks; the second verifier’s Part F prints “= the published bound” for $35 + 9$, a phrasing its own analysis retracts (§6, item 5).
9. **The prose reports written during the work disagree with the code in several places** — check counts, runtimes, one shape’s class label, and aggregate network totals. All are itemised in `NOTES.md`; the code is authoritative and this paper follows the code.

10. Discussion

10.1 How an error survives fifty-four years

Three things had to coincide. First, the chapter’s single worked example is clean, so it cannot expose the false premise; and its Kraft sum lands at 0.75, below 1, so the one visible symptom is on the harmless side. Second, the false step is a *citation*: the structural claim “N-1 branch nodes (the comparators)” is attributed to another paper, and a reader checking the chapter reasonably treats it as imported. Third — and decisively — nobody read it. Every modern citation of the chapter is for the one-value bound, which lives in a different paper; the two-value argument has, as far as we can determine, no readers at all in the literature (§6).

The fourth ingredient is recent: a fifty-four-year-old unread argument was combined with a 2020 computational result and posted to a widely consulted table. The recombination is arithmetically correct and the input is not.

10.2 What the finding is worth

Narrowly: it removes one from a table entry that no publication depends on. More usefully, it identifies a concrete, self-contained open problem with a known answer in one instance ($P(2,11) = 9$, now reduced to a single tree shape), a proved special case (Theorem 6), a precise obstruction (pass-throughs), and a formalisation target that is small enough to be worth attempting: Facts 1–2, (R1)–(R3), Lemmas 1–5 and Theorem 6 involve no arithmetic beyond Kraft’s inequality and would transcribe to Isabelle or Lean directly. That is what we would formalise first, not because the clean case is the important one, but because it fixes the vocabulary — red/blue leads, branch tree, pass-through — that any general proof will need.

10.3 Methodology

This audit was carried out by a set of language-model agents operating under a fixed protocol: one agent read the primary source and produced the first audit; a second was instructed to defend the chapter and refute the first agent’s finding, working from the page images and forbidden to import the first agent’s code; a third attempted repairs and refuted its own candidate lemma. The division of labour was useful mainly because the adversarial role produced the counterexample T3 that killed the per-node repair family, and because the independent re-implementation reproduced every disputed number under a separate codebase. We claim no methodological result from this. What matters for the reader is that no mathematical claim in this paper rests on a model’s assertion: every quantitative statement is produced by one of the two scripts of §9, both of which are short, dependency-free, and readable by hand, and the central counterexample is a three-comparator network that can be checked on paper in a minute. Where the prose reports produced during the work disagree with the

scripts, we have taken the scripts as authoritative and recorded the discrepancies in `NOTES.md` rather than silently reconciling them.

11. Limitations

1. **Two archival items were not obtained, and one of them may be the origin of the error.**

- **Van Voorhis, “An improved lower bound for sorting networks”, IEEE Trans. Computers C-21(6) (1972), 612–613.** This is [Van Voorhis (1972A)], the reference the chapter cites *specifically* for the claim that the MAX subnetwork is a binary tree with $N-1$ branch nodes (the comparators). The two-page full text is paywalled and we did not obtain it. **What it could change:** if the note states the structural claim correctly — for instance with a cleanliness hypothesis, or in terms of branch nodes rather than comparators — then the chapter’s error is a transcription error rather than an inherited one, and the note may contain the missing argument or the hypothesis under which it is true. If it states the claim in the same false form, the error originates there and propagates to a paper (the one-value bound) that *is* peer-reviewed and widely cited — though we note that Knuth’s independent three-line proof of the one-value bound does not depend on it, so $S(13) \geq 43$ would be unaffected either way.
- **Van Voorhis, “Efficient Sorting Networks”, Ph.D. dissertation, Stanford University, 1971.** Not digitised. **What it could change:** if a fuller treatment of the two-value argument exists anywhere, this is the remaining place to look; it could contain the antichain argument the chapter omits, a hypothesis restricting the class of networks, or the proof of $P(2,11) = 9$ that the chapter declines to give. It could in principle close the gap entirely, in which case the correct finding would be that the *chapter’s* exposition is invalid but the theorem is proved elsewhere.

Obtaining either item is the single highest-value next step and should precede any attempt to publish.

2. **Theorem 6 is not machine-checked as a proof.** It is a human proof whose lemmas and conclusion are tested on samples (checks **V1:G1–V1:G4**). No proof assistant has verified it. Given that §7.6.1 documents a lemma that survived 900 samples and was then refuted, sample agreement is weak evidence and we do not offer it as more.

3. **The population of “constructed sorters” is not a uniform sample of anything.** The generators are Batcher, bubble, insertion, odd-even transposition, and random prefixes thinned against redundancy. Prevalence percentages (61.0 %, 42.4 %, 27.1 %) are properties of *these generators*, not of sorting networks in any measure-theoretic sense. They establish that the failure is common

in ordinary networks; they do not establish a rate.

4. $S(13) \geq 44$ is not refuted. Nothing here suggests it is false. Roughly 15,000 networks including a search aimed directly at the statement failed to break it. Our claim is exactly and only about the status of its proof.

5. Two of the closed-off routes in §7.6.3 are analytic, not machine-checked (the fixed-channel Kraft route yielding 43, and the pair-counting route yielding 156). A referee is entitled to ask for these in full; they are elementary but they are not in the artifact.

6. The artifact has known defects that would need fixing before release. §9.5 lists nine. Two matter for the claims: check D7 asserts nothing, and checks D3/D6 establish for one shape what the surrounding prose asserts for six. None of them affects the counterexample, the prevalence figures, or Theorem 6, but a referee reading the code will find them and a released artifact should not contain them.

7. Secondary citations not verified against a primary source in the artifact. The attributions for $S(9) = 25$, $S(10) = 29$, the Juillé 1995 upper bound of 45, and the exact wording and date of Dobbelaere’s changelog entry are carried from project notes and background literature; only the 1972 chapter, the STAN-CS-71-238 technical report, the STOC ’95 asymptotic lower-bound paper, and the 2025 depth paper are physically present in *papers/*. These should be checked before any external use.

12. References

[VV72] D. C. Van Voorhis, “Toward a Lower Bound for Sorting Networks”, in R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, The IBM Research Symposia Series, Plenum Press, New York, 1972, pp. 119–129. (*In papers/; PDF pages 124–134. All equation numbers, Theorem 1, Table 1 and Figs. 1–5 in this paper are the chapter’s own; page numbers are book pages.*)

[VV72a] D. C. Van Voorhis, “An improved lower bound for sorting networks”, *IEEE Transactions on Computers* C-21(6) (1972), 612–613. **Not obtained** — see §11.

[VV71a] D. C. Van Voorhis, “Efficient Sorting Networks”, Ph.D. dissertation, Stanford University, 1971. **Not obtained** — see §11.

[VV71b] D. C. Van Voorhis, “A Lower Bound for Sorting Networks that Use the Divide-Sort-Merge Strategy”, Stanford Digital Systems Laboratory Technical Report No. 17, STAN-CS-71-238 / SEL-71-051, August 1971. (*In papers/. Restricted-class bound, $\Theta(N (\log_2 N)^2)$; unrelated to the two-value theorem.*)

- [Kn73] D. E. Knuth, *The Art of Computer Programming*, Vol. 3: *Sorting and Searching*, §5.3.4; exercise 42 (p. 240) and its answer (p. 671).
- [FK73] R. W. Floyd and D. E. Knuth, “The Bose–Nelson sorting problem”, in *A Survey of Combinatorial Theory*, 1973. (*Source of $S(n)$ for $n \leq 8$; cited by [VV72] as Floyd and Knuth (1970A).*)
- [Gr70] M. W. Green, cited by [VV72] as Green (1970A), for the pruning idea (p. 120) and for the $U(N)$ column of Table 1.
- [Ba68] K. E. Batcher, “Sorting networks and their applications”, AFIPS Spring Joint Computer Conference, 1968. (*Cited by [VV72] as Batcher (1968A).*)
- [CCFS14] M. Codish, L. Cruz-Filipe, M. Frank and P. Schneider-Kamp, “Twenty-five comparators is optimal when sorting nine inputs (and twenty-nine for ten)”, arXiv:1405.5754.
- [Ha20] T. Harder, “An Answer to the Bose–Nelson Sorting Problem for Eleven and Twelve Elements” (2020), and the accompanying `sortnetopt` development. (*Source of $S(11) = 35$, $S(12) = 39$; contains no reference to Van Voorhis.*)
- [Ju95] H. Juillé, “Evolution of non-deterministic incremental algorithms as a new approach for search in state spaces”, 1995. (*45-comparator 13-sorter; still the best known upper bound.*)
- [KLM+95] N. Kahale, T. Leighton, Y. Ma, C. G. Plaxton, T. Suel and E. Szemerédi, “Lower Bounds for Sorting Networks”, STOC ’95. (*In **papers**/. Asymptotic: $(1.12 - o(1)) n \log n$ for size. Vacuous at $n = 13$; cited here only as evidence that asymptotic machinery does not descend to this range.*)
- [Do25] B. Dobbelaere, “Smallest and fastest sorting networks for a given number of inputs”, online table; changelog entry of 2025-04-21.
- [Wa25] C. Wang, “Depth-13 Sorting Networks for 28 Channels”, arXiv:2511.04107v2. (*In **papers**/. A **depth** result for $n = 27, 28$; no bearing on $S(13)$. Recorded to forestall the name collision.*)